MUSE

Find your inspiration.
Share your creation.

Warm Ivory	Soft Sage	Terracotta	Sand Beige	Deep Olive
#F7F4EC	#A8B5A0	#C8795A	#DCCBB8	#343B35

PRODUCT REQUIREMENTS DOCUMENT

Complete project brief for design, development, Supabase integration and browser-based testing.

Version: 1.0 **Date:** 8 November 2026 **Project:** MUSE

Build approach: Plain HTML, CSS and Vanilla JavaScript with Supabase. Designed for local preview using VS Code Live Server.

1. Product overview

Product name: MUSE

Meaning: A muse is a source of inspiration. The name represents a place where creative people find ideas, share their work and inspire one another.

Tagline: Find your inspiration. Share your creation.

Product category: Creative social networking platform.

Product vision: Build a welcoming online space where creators can present their work, discover new ideas, receive constructive feedback and form meaningful creative connections.

Target audience

- Graphic designers and UI/UX designers.
- Web developers and software students.
- Photographers, illustrators and digital artists.
- Students, hobbyists and creative beginners.
- Independent creators looking for inspiration, feedback and collaboration.

Core product principles

- **Creativity first:** Give projects and ideas the main stage.
- **Community over popularity:** Encourage meaningful interaction, not only follower counts.
- **Welcoming by design:** Make it easy for beginners to participate.
- **Clear ownership:** Users control and manage their own profiles and projects.
- **Trust and privacy:** Protect user data with authentication and database access policies.

2. Goals and success criteria

Primary goals

- Allow users to register, log in and manage a creator profile.
- Allow creators to publish, edit, delete and save project drafts.
- Help users discover projects and creators through search and categories.
- Support likes, comments, follows and bookmarks.
- Provide creative challenges and project submissions.
- Show useful notifications for community activity.
- Provide a personal portfolio view for each creator.

Acceptance criteria

- All 16 planned pages load and navigate correctly.
- The layout is responsive on desktop, tablet and mobile.
- Authentication and data operations use Supabase, not simulated success messages.
- Users can only modify records they own.

- Forms show validation, loading, success and error states.
- No secret service-role key is exposed in browser code.
- The project can be previewed locally without a framework or build step.

3. Brand and design system

Visual direction: **Warm Creative Studio** — friendly, editorial, expressive and polished. The interface should feel like a thoughtfully curated creative community, not a generic admin dashboard.

Token	Value	Usage
Warm Ivory	#F7F4EC	Main page backgrounds and spacious sections
Soft Sage	#A8B5A0	Primary buttons, selected states and calm accents
Terracotta	#C8795A	Highlights, badges and important calls to action
Sand Beige	#DCCBB8	Borders, secondary panels and soft surfaces
Deep Olive	#343B35	Primary text, navigation and strong contrast

Typography and components

- **Headings:** Playfair Display.
- **Body text:** DM Sans.
- **Cards:** Rounded corners, subtle shadows, consistent spacing and clear image hierarchy.
- **Buttons:** Sage as the primary action color; terracotta as a restrained accent.
- **Navigation:** Consistent brand mark, page links and account actions.
- **Motion:** Gentle hover transitions and subtle entrance effects; respect reduced-motion preferences.
- **Accessibility:** Semantic HTML, visible focus states, useful labels, keyboard-friendly controls and readable contrast.

Avoid

- Excessive gradients, overly dark full-page backgrounds and visual clutter.
- Tiny text, low-contrast labels and hover-only interactions.
- Unrelated styles between pages or inconsistent navigation.
- Fake buttons that look functional but do nothing.

4. Technical architecture

Technology	Role / constraint
HTML5	Semantic page structure; one HTML file per page.
CSS3	Theme, layouts, reusable components and responsive styling.
Vanilla JavaScript	Page behavior, validation, navigation and Supabase operations.
Supabase JS SDK v2	Browser client for authentication and permitted database/storage operations.
Supabase Auth	Signup, login, logout, password reset and session handling.
Supabase PostgreSQL	Persistent relational application data.
Supabase Storage	Creator avatars and project images.

Technology	Role / constraint
Supabase Realtime	Optional live notification updates where appropriate.
VS Code Live Server	Local browser preview during development.
Vercel	Static-site deployment after local testing.

Strict stack rules

Use plain HTML, CSS and Vanilla JavaScript only. Do not introduce React, Next.js, Vue, Angular, JSX or TypeScript. Bootstrap 5 may be used selectively for layout utilities, but custom MUSE styling must remain consistent. No build tool or Node.js setup should be required for the local preview.

5. Page-by-page requirements

#	Page / file	Purpose and key requirements
1	Landing — index.html	Brand introduction, hero section, platform benefits, featured work, creator highlights and clear Sign Up / Explore calls to action.
2	Sign Up — signup.html	Name, username, email, password, confirmation, terms acceptance, validation and Supabase registration.
3	Login — login.html	Email/password login, show/hide password, reset-password flow, session handling and helpful errors.
4	Discover — discover.html	Featured projects, trending creators, popular categories, new work and curated highlights.
5	Home Feed — feed.html	Recent published projects, creator context, like/comment/bookmark actions and feed empty/loading states.
6	Explore — explore.html	Search projects and creators; filter by category, tools, newest or relevant results.
7	Project Details — project-details.html	Project cover/gallery, title, description, creator link, tools, external URL, likes and comments.
8	Create Project — create-project.html	Project form, image upload, category, tools, external URL, save draft and publish actions.
9	My Portfolio — portfolio.html	Current user's published projects, portfolio presentation and project management links.
10	Creator Profile — profile.html	Public creator identity, avatar, bio, skills, social links, projects, follow action and counts.
11	Edit Profile — edit-profile.html	Update full name, username, bio, skills, avatar and social links.
12	Notifications — notifications.html	Likes, comments, follows and challenge-related activity; read/unread states.
13	Challenges — challenges.html	Active, upcoming and completed challenges with category and deadline information.
14	Challenge Details — challenge-details.html	Challenge brief, rules, dates, submission action and participant submissions.
15	Saved Projects — saved.html	Current user's bookmarked projects with remove-bookmark action.
16	Settings — settings.html	Account preferences, privacy options, password/security links and logout.

Shared interface requirements

- A consistent responsive navbar and footer across applicable pages.
- Clear active navigation state and working links.
- Authentication-aware navigation: logged-out and logged-in states.
- Reusable project cards, creator cards, form controls and notification items.
- Confirmation before destructive actions such as deleting a project.
- Useful empty states when a user has no projects, bookmarks or notifications.

6. Core functional requirements

Authentication and profiles

- Register and sign in through Supabase Auth.
- Create a matching profile record after successful registration.
- Support logout, password reset and session restoration.
- Protect pages/actions that require authentication.
- Allow users to edit only their own profile.

Projects and portfolio

- Create projects with title, description, category, tools, cover image and optional external link.
- Save drafts and publish projects.
- Edit or delete only projects owned by the signed-in user.
- Show only published projects publicly; show a user's own drafts to that user.
- Allow multiple project images and ordered display.

Community interactions

- Like and unlike projects; prevent duplicate likes.
- Add comments and allow their authors to edit or delete them.
- Follow and unfollow creators; prevent self-follow and duplicate follows.
- Bookmark and remove bookmarks; bookmarks remain private to their owner.
- Generate notifications through trusted logic when meaningful events occur.

7. Supabase data model

The following tables form the planned relational model. The coding agent must generate the final SQL migration scripts, constraints, indexes, triggers/functions where needed, RLS policies and Storage policies after reviewing the completed implementation.

Table	Important fields / purpose
profiles	User ID linked to auth.users; username, full name, bio, avatar URL, skills, website and social links.
projects	Owner, title, description, category, tools, cover image, external URL, draft/published status and timestamps.
project_images	Project reference, image URL and display order.
likes	User and project relationship; unique pair prevents duplicate likes.
comments	Author, project, comment content and timestamps.
follows	Follower and following profile IDs; unique pair and no self-follow.
bookmarks	User and saved project relationship; unique pair.
challenges	Title, description, category, cover, start date, deadline and status.
challenge_submissions	Challenge, creator, submitted project and description.
notifications	Recipient, actor, event type, related project/comment, message, read state and timestamp.

Security and data rules

- Enable Row Level Security on every user-related table.
- Public users may read only public profile data and published projects.
- Users may create/update/delete only records they own, subject to the feature's rules.
- Bookmarks and notifications must be visible only to their owner/recipient.
- Do not allow browser clients to create arbitrary notifications for other users.
- Validate project ownership before accepting images or challenge submissions.
- Use Supabase Storage buckets for avatars and project media, with owner-scoped upload policies.
- Never expose the Supabase service_role key in client-side HTML or JavaScript.

Storage plan

Bucket	Contents	Policy direction
muse-avatars	Profile pictures	Authenticated users upload/update files only in their own user-specific folder.
muse-projects	Project covers and gallery images	Authenticated project owners upload media; public reads only for media attached to published projects where intended.

8. Search, discovery and moderation

- Search project titles, descriptions, categories and creator usernames.

- Offer useful category filters such as Graphic Design, UI/UX, Web Development, Photography, Illustration and Digital Art.
- Provide sorting options such as newest and popular, with clear definitions.
- Do not expose drafts in public search or feeds.
- Plan for reporting inappropriate projects/comments and basic moderation controls before public launch.
- Validate text length, image type/size and external links.

9. Privacy, security and accessibility

- Use HTTPS in deployment and Supabase's public client key only in the browser.
- Enforce data access in Supabase RLS; hiding buttons in the UI is not security.
- Do not store plaintext passwords or sensitive secrets in localStorage.
- Use safe DOM rendering for user-generated content; avoid unsafe HTML insertion.
- Add labels, alt text, keyboard access and visible focus styles.
- Provide confirmation and clear feedback for important account/project actions.
- Add a privacy policy and community guidelines before public launch.

10. Recommended folder structure

```
MUSE/
■■■ index.html
■■■ signup.html
■■■ login.html
■■■ discover.html
■■■ feed.html
■■■ explore.html
■■■ project-details.html
■■■ create-project.html
■■■ portfolio.html
■■■ profile.html
■■■ edit-profile.html
■■■ notifications.html
■■■ challenges.html
■■■ challenge-details.html
■■■ saved.html
■■■ settings.html
■■■ css/
■   ■■■ style.css
■   ■■■ auth.css
■   ■■■ dashboard.css
■   ■■■ projects.css
■   ■■■ responsive.css
■■■ js/
■   ■■■ supabase.js
■   ■■■ auth.js
■   ■■■ common.js
■   ■■■ projects.js
■   ■■■ feed.js
■   ■■■ profile.js
■   ■■■ interactions.js
■   ■■■ challenges.js
■   ■■■ notifications.js
■■■ assets/
    ■■■ images/
    ■■■ icons/
    ■■■ logo/
```

The coding agent may add small, clearly named files when genuinely needed, but it must preserve the plain HTML/CSS/JavaScript architecture and explain any structural change.

11. Development phases

Phase	Work	Completion checkpoint
1	Initialize folders, design tokens, shared navigation/footer and landing page.	Landing page previews locally; responsive layout and links work.
2	Signup, login, logout, password reset and session handling.	Supabase Auth flow tested with real project credentials.
3	Profile setup, public profile and edit profile.	Profile data loads and updates with ownership checks.
4	Create project, image upload, drafts and portfolio.	CRUD and Storage upload tested against Supabase.
5	Feed, discover, explore, search and project details.	Published content loads; filters and project navigation work.
6	Likes, comments, follows and bookmarks.	Interactions persist and duplicate actions are prevented.

Phase	Work	Completion checkpoint
7	Challenges, submissions and notifications.	Challenge flow and notification access are tested.
8	Settings, privacy, accessibility and responsive QA.	All pages and important flows pass testing.
9	Final database scripts, Storage policies, deployment and handover.	Agent provides exact Supabase setup steps and final checklist.

12. Local development and preview

- Open the MUSE folder in VS Code.
- Install/use the Live Server extension.
- Open index.html with Live Server to preview the site in a browser.
- Use relative file paths so pages work when opened from the project folder.
- Test at desktop, tablet and mobile widths after each page.
- Use public image URLs during early layout work; migrate uploads to Supabase Storage when that feature is implemented.

13. Coding-agent working agreement

- Build one page at a time; do not generate all 16 pages in one uncontrolled step.
- Before each new page, inspect existing files and reuse the established theme/components.
- Do not unexpectedly redesign existing pages or remove working features.
- Keep HTML, CSS and JS in separate files.
- Explain each changed/created file and provide exact local preview instructions.
- Test navigation, forms, responsive behavior and console errors before declaring a page complete.
- Do not claim a Supabase feature works until it has been tested with a connected project.

14. Supabase handover — important instruction

The user wants the coding agent to prepare the Supabase setup instructions at the end, after the application code and database design are ready. Do not ask the user to run SQL commands now.

- At the final integration stage, the coding agent must provide one complete, reviewed SQL script (or clearly ordered migration scripts) for all required tables, relationships, indexes, constraints, triggers/functions and RLS policies.
- The agent must separately provide Storage bucket creation and upload/read policy instructions.
- The agent must list the exact Supabase Dashboard steps and identify where the project URL and anon/publishable key go.
- The agent must clearly identify which actions the user must perform manually in Supabase and in what order.
- The agent must never ask the user to paste a service_role key into frontend code.
- The user will apply the supplied commands/instructions only after the agent has provided them, then share any errors for troubleshooting.

15. Final deliverables

- Complete MUSE source folder with all implemented pages and assets.
- Consistent responsive Warm Creative Studio design.
- Working Supabase authentication, database operations and image storage.
- Final SQL/migration scripts and RLS policies.
- Storage bucket and policy setup guide.
- README with Live Server instructions and deployment notes.
- Testing checklist with known limitations clearly documented.

End of MUSE PRD • Version 1.0